Mojo: ООП, обобщённые типы и обработка ошибок

- Григорьева А.
- Шевцов Д.
- Шильдт Д.

5030102/30202
11-я команда

План

01 Объектно-, функционально-, протокол-ориентированное программирование в Mojo

02 Обобщённые типы (generics)

03 Обработка ошибок

04 Управление сложностью кода (модули, пакеты)

Объектно-ориентированный подход: struct

Альтернатива классам

В Mojo нет классов и наследования, как в Python — вместо этого **struct**

Данные и поведение

struct объединяет данные (**var**) и поведение (методы)

Переиспользование через Traits

Классическое наследование отсутствует — используйте **traits**

Инкапсуляция

Пока по соглашению: `private/public` ещё нет (обсуждается как фича), скрытые поля помечают префиксом _ (например, `_scale_count`)

```mojo
struct Rectangle:
    var width: Float64
    var height: Float64
    var _scale_count: Int

    def __init__(out self, width: Float64, height: Float64):
        self.width = width
        self.height = height
        self._scale_count = 0

    def area(self) -> Float64:
        return self.width * self.height

    def scale(mut self, factor: Float64):
        self.width *= factor
        self.height *= factor
        self._scale_count += 1

def main():
    var r = Rectangle(3.0, 4.0)
    print("Площадь:", r.area())
    r.scale(2.0)
    print("После scale:", r.area())
    print("Вызовов scale:", r._scale_count)
```

Trait — контракт поведения

Определение контракта

trait описывает контракт: какие методы обязан иметь тип

Объявление соответствия

struct явно объявляет соответствие трейту в скобках

Статическая проверка

Соответствие проверяется при компиляции — без duck typing

Аналоги в других языках

Protocols (Swift/Python), interfaces (Java/Go), traits (Rust)

```
trait Shape:
    def area(self) -> Float64:
        ...
    def perimeter(self) -> Float64:
        ...

struct Rectangle(Shape):
    var width: Float64
    var height: Float64

    def __init__(out self, width: Float64, height: Float64):
        self.width = width
        self.height = height

    def area(self) -> Float64:
        return self.width * self.height

    def perimeter(self) -> Float64:
        return 2 * (self.width + self.height)


def describe[T: Shape](shape: T):
    print("Площадь:", shape.area(), "Периметр:", shape.perimeter())


def main():
    var r = Rectangle(3.0, 4.0)
    describe(r)
```

Refinement и композиция трейтов

Наследование трейтов (refine)

Trait может наследовать другой trait — расширяет контракт

Композиция трейтов

Несколько трейтов комбинируются через & в одном ограничении

Функции первого класса

Значения первого класса (можно передавать как аргументы) — функциональная часть парадигмы

```
trait Shape:
    def area(self) -> Float64:
        ...

trait NamedShape(Shape):   # наследует Shape
    def name(self) -> String:
        ...

struct Circle(NamedShape):
    var radius: Float64
    def area(self) -> Float64:
        return 3.14159 * self.radius ** 2
    def name(self) -> String:
        return "Circle"

def print_info[T: Shape & Copyable](shape: T):  # композиция трейтов
    print(shape.area())
```

```
def double(x: Int) -> Int:
    return x * 2

def apply[F: def(Int) -> Int, //](f: F, x: Int) -> Int:
    return f(x)

def main():
    print(apply(double, 5))   # 10
```

Параметризованные функции: [] vs ()

Параметры типа []

В квадратных скобках **[ ]**, вычисляются на этапе компиляции

Ограничения типов

Тип **обязательно** ограничивается трейтом (нельзя оставить без ограничений)

Аргументы ()

В круглых скобках **( )**, вычисляются во время выполнения

Специализация кода

Компилятор генерирует отдельную версию функции под каждый конкретный тип

```
1  def max_value[T: Comparable & Copyable](a: T, b: T) -> T:
2      if a > b:
3          return a.copy()
4      return b.copy()
5
6  def main():
7      print(max_value(3, 7))        # T выведен как Int
8      print(max_value("a", "b"))    # T выведен как String
9      print(max_value(3.0, 4))      # T выведен как Float64, 4 → 4.0
10     # print(max_value(4, 3.0))      ошибка компиляции: T уже Int, Float → Int неявно не работает
```

Параметризованные типы: Pair[T]

Параметризация структур

Структуры тоже параметризуются типом — в `[ ]` после имени структуры

Поля и методы

Один параметр типа может использоваться и для полей, и для методов

Указание типа

Явно: `Pair[Int]` или выводится компилятором

```
struct Pair[T: Comparable & Copyable]:
    var left: Self.T
    var right: Self.T

    def __init__(out self, left: Self.T, right: Self.T):
        self.left = left.copy()
        self.right = right.copy()

    def max(self) -> Self.T:
        if self.left > self.right:
            return self.left.copy()
        return self.right.copy()


def main():
    var p = Pair[Int](3, 7)
    print(p.max())
```

Гибкие ограничения на типы

`Some[Trait]`

Короткая запись вместо явного параметра типа в `[]`

Условие `where`

Дополнительное условие, которое компилятор обязан доказать перед вызовом

Управление возможностями

Позволяет включать/выключать методы или соответствие трейту по условию

```
def show(x: Some[Writable]):
    print(x)


struct Box[T: Copyable]:
    var value: Self.T

    def print_if_writable(self) where conforms_to(Self.T, Writable):
        print(self.value)
```

Обработка ошибок в Mojo: не исключения, а значения

Возвращаемое значение

Ошибка — альтернативное возвращаемое значение, а не разворачивание стека (как в C++/Java)

Объявление `raises`

Функция обязана явно объявить `raises`, если может выбросить ошибку

Тип `Error`

Базовый тип ошибки - `Error`, создаётся из строки

```mojo
def divide(a: Int, b: Int) raises -> Int:
    if b == 0:
        raise Error("деление на ноль")
    if b < 0:
        raise Error("делитель не может быть отрицательным")
    return a // b

def average(a: Int, b: Int, c: Int) raises -> Int:
    return divide(a + b + c, 3)

def halve(a: Int) -> Int:
    return a // 2
```

Перехват ошибок и проверки на этапе выполнения

try / except / else / finally

`try/except` — перехват; `else` — без ошибок; `finally` — всегда

Проверки debug_assert

Проверка условия в рантайме для отладки (в отличие от `raise` — не часть контракта функции)

Релизная сборка

`debug_assert` по умолчанию **выключен** в релизе — не заменяет обработку ошибок

```
var result: Int
try:
    result = divide(10, 0)
except e:
    print("Ошибка:", e)
else:
    print("Успех:", result)
finally:
    print("Готово")

debug_assert(b != 0, "b не должен быть нулём")
```

Управление сложностью: модули

Файл `.mojo`

Модуль в Mojo — это один файл `.mojo`, который можно импортировать в другой

Импорт `from` и `import`

Импорт конкретного имени (`from module import X`) или всего модуля (`import module`)

Псевдонимы `as`

Переименование модуля или сущностей при импорте для удобства доступа и избежания конфликтов

```mojo
# mymodule.mojo
struct MyPair:
    var first: Int
    var second: Int


# main.mojo
from mymodule import MyPair
from mymodule import MyPair as Pair
import mymodule as m

def main():
    var p = MyPair(2, 4)
    var p3 = Pair(5, 6)
    var p2 = m.MyPair(1, 1)
```

Пакеты: `__init__.mojo`

Структура пакета

Директория с файлом **`__init__.mojo`** (обязателен, может быть пустым)

Имя и ре-экспорт

Имя директории = имя пакета. Позволяет ре-экспортировать члены для прямого доступа

Компиляция в `.mojoc`

Возможность скомпилировать пакет для распространения без исходного кода

```
main.mojo
mypackage/
    __init__.mojo
    mymodule.mojo
```

```mojo
# mypackage/__init__.mojo
from .mymodule import MyPair

# main.mojo
from mypackage import MyPair              # через ре-экспорт
# from mypackage.mymodule import MyPair
# или напрямую, по полному пути — тоже сработает

def main():
    var p = MyPair(2, 4)
```

```
mojo precompile mypackage -o mypack.mojoc
from mypack import MyPair
```